Mini Projet en programmation C

SHOPFLOW — GESTION DES COMMANDES D'UNE BOUTIQUE EN LIGNE

Section 1 - Présentation du Projet

Ce mini-projet doit être réalisé en binôme. Il a pour objectif de développer un système de gestion informatisé pour une boutique de commerce en ligne, en appliquant les concepts de programmation modulaire et d'allocation dynamique en langage C.

L'application devra permettre de gérer le catalogue des produits, les clients et leurs commandes, ainsi que d'établir des statistiques d'activité (chiffre d'affaires, meilleures ventes, meilleurs clients). Contrairement à un exercice classique, le nombre de produits, de clients et de lignes par commande n'est pas connu à l'avance : c'est la contrainte structurante du projet. Les collections devront donc être allouées, redimensionnées et libérées dynamiquement (malloc, realloc, free).

Le projet mettra l'accent sur une architecture modulaire claire et bien structurée, favorisant la maintenance, la réutilisabilité et l'évolutivité du code. Chaque fonctionnalité devra être implémentée dans un module dédié, en respectant les bonnes pratiques de programmation en C.

La présentation du projet fera l'objet d'une soutenance de 20 minutes lors de la dernière séance du module XTI101, prévue durant la semaine du 2 novembre 2026. L'ordre de passage des binômes sera communiqué ultérieurement.

Section 2 - Règles Métier de la Boutique

La boutique applique une politique de remise commerciale progressive sur le montant hors taxes de chaque commande :

Montant HT de la commande	Remise appliquée
Inférieur à 100 €	0 %
De 100 € à 299,99 €	5 %
De 300 € à 599,99 €	10 %
Supérieur ou égal à 600 €	15 %

Le palier de remise est déterminé sur le montant HT brut de la commande (somme des sous-totaux des lignes), avant application de la remise.

Règles de facturation à respecter :

- TVA : 20 % appliquée sur le montant HT après remise. Les frais de port ne sont pas soumis à la TVA dans ce projet.
- Frais de port : 5,90 €, offerts à partir de 60 € d'achat HT brut (avant remise).
- Fidélité : 1 point attribué par tranche de 10 € payée TTC (port inclus), crédités à la création de la commande et retirés en cas d'annulation.

- Statuts possibles d'une commande : EN_COURS, EXPEDIEE, LIVREE, ANNULEE. Progression autorisée : EN_COURS → EXPEDIEE → LIVREE, sans retour possible ; l'annulation n'est possible qu'au statut EN_COURS.

L'application doit produire deux classements distincts :

- Palmarès produits : classement des produits par quantité totale vendue.
- Palmarès clients : classement des clients par chiffre d'affaires généré. Le chiffre d'affaires et les deux palmarès excluent les commandes au statut ANNULEE ; les quantités vendues ne portent que sur les lignes de commandes non annulées.

Section 3 - Modélisation des Données

Cette section détaille les 7 structures principales qui constituent l'architecture de votre application. Chaque structure est présentée avec un tableau détaillé de ses attributs et de leurs contraintes de validation.

3.1 Structures de données

3.1.1 Structure Date

Attribut	Type	Contraintes	Description	Exemple
jour	int	1 ≤ jour ≤ 31	Jour du mois (validé selon le mois)	12
mois	int	1 ≤ mois ≤ 12	Mois de l'année (janvier = 1)	3
annee	int	2000 ≤ annee ≤ 2100	Année de la commande (gestion bissextile)	2026

3.1.2 Structure Adresse

Attribut	Type	Contraintes	Description	Exemple
rue	char[80]	Non vide, 79 car. max	Numéro et libellé de voie	12 rue de la Paix
ville	char[50]	Non vide	Ville de livraison	Paris
codePostal	char[6]	5 chiffres	Code postal	75002
pays	char[50]	Non vide	Pays de livraison	France

3.1.3 Structure Produit

Attribut	Type	Contraintes	Description	Exemple
reference	char[12]	Unique, non vide	Référence commerciale (SKU)	PRD-0001
designation	char[60]	Non vide	Libellé du produit	Clavier mécanique
categorie	char[30]	Non vide	Famille de produit	Informatique
prixUnitaireHT	float	> 0	Prix unitaire hors taxes	79.90
quantiteStock	int	≥ 0	Stock disponible	42
seuilAlerte	int	≥ 0	Seuil de réapprovisionnement	10
actif	int	0 ou 1	Statut du produit (suppression logique)	1

3.1.4 Structure LigneCommande

Attribut	Type	Contraintes	Description	Exemple
refProduit	char[12]	Doit exister au catalogue	Référence du produit commandé	PRD-0001
designation	char[60]	Cohérente avec le catalogue	Libellé figé au moment de la vente	Clavier mécanique

Attribut	Type	Contraintes	Description	Exemple
quantite	int	1 ≤ qté ≤ stock dispo.	Quantité commandée	2
prixUnitaireHT	float	> 0	Prix figé au moment de la vente	79.90
sousTotalHT	float	≥ 0, calculé	quantite × prixUnitaireHT	159.80

3.1.5 Structure Commande

Attribut	Type	Contraintes	Description	Exemple
numero	int	Unique, > 0	Numéro de commande	1001
idClient	int	Doit exister	Référence du client propriétaire	3
dateCommande	Date	Structure imbriquée	Date de validation de la commande	{12, 3, 2026}
statut	char[20]	4 valeurs autorisées	État d'avancement de la commande	EXPEDIEE
nbLignes	int	1 ≤ nb ≤ capacite	Nombre de lignes réellement saisies	3
capaciteLignes	int	≥ nbLignes	Taille du tableau alloué	4
lignes	LigneCommande *	Alloué dynamiquement	Tableau de lignes de commande	malloc
montantBrutHT	float	≥ 0, calculé	Somme des sous-totaux des lignes, avant remise	285.96
tauxRemise	float	0, 5, 10 ou 15	Taux du palier, déterminé sur le montant HT brut	5.00
montantRemise	float	≥ 0, calculé	montantBrutHT × tauxRemise / 100	14.30
montantHT	float	≥ 0, calculé	montantBrutHT – montantRemise	271.66
montantTVA	float	≥ 0, calculé	20 % du montant HT	54.33
montantTTC	float	≥ 0, calculé	HT + TVA + frais de port	331.89
fraisPort	float	0 ou 5.90	Frais de port appliqués (offerts dès 60 € HT brut)	5.90
adresseLivraison	Adresse	Recopiée à la création	Instantané de l'adresse de livraison au moment de la commande	{12 rue de la Paix, …}

3.1.6 Structure Client

Attribut	Type	Contraintes	Description	Exemple
idClient	int	Unique, > 0	Identifiant du client	3
nom	char[50]	Non vide, 49 car. max	Nom de famille	Bernard
prenom	char[50]	Non vide	Prénom	Julie
email	char[80]	Contient « @ », unique	Adresse électronique	j.bernard@mail.fr
telephone	char[15]	10 chiffres	Téléphone de contact	0612345678
adresse	Adresse	Structure imbriquée	Adresse de livraison	{12 rue de la Paix, …}
pointsFidelite	int	≥ 0, calculé	Points de fidélité cumulés	34
nbCommandes	int	≥ 0	Nombre de commandes du client	2
capaciteCommandes	int	≥ nbCommandes	Taille du tableau alloué	2
commandes	Commande *	Alloué dynamiquement	Historique des commandes	malloc
actif	int	0 ou 1	Statut du client (suppression logique)	1

3.1.7 Structure Boutique (structure racine)

Attribut	Type	Contraintes	Description	Exemple
nom	char[50]	Non vide	Nom commercial de la boutique	ShopFlow
produits	Produit *	Alloué dynamiquement	Catalogue des produits	malloc
nbProduits	int	≥ 0	Nombre de produits au catalogue	8
capaciteProduits	int	≥ nbProduits	Capacité allouée du catalogue	10
clients	Client *	Alloué dynamiquement	Répertoire des clients	malloc
nbClients	int	≥ 0	Nombre de clients enregistrés	5
capaciteClients	int	≥ nbClients	Capacité allouée du répertoire	8
numeroSuivant	int	> 0	Compteur d'attribution des numéros de commande	1007
idClientSuivant	int	> 0	Compteur d'attribution des identifiants clients	6

3.2 Schéma des Relations entre Structures

```
Boutique  (structure racine, possedee par main.c)

|-- Produit *produits            (tableau dynamique, capacite variable)

|     `-- reference  <-- referencee par LigneCommande.refProduit

`-- Client *clients              (tableau dynamique, capacite variable)

      |-- Adresse adresse        (structure imbriquee)

      `-- Commande *commandes    (tableau dynamique, capacite variable)

            |-- Date dateCommande      (structure imbriquee)

            `-- LigneCommande *lignes  (tableau dynamique, capacite variable)

                  `-- sousTotalHT  -->  alimente Commande.montantHT
```

Les quatre niveaux d'allocation dynamique : Boutique, Client, Commande et LigneCommande.

Relations clés à implémenter :

1. **Composition :** Client contient une Adresse ; Commande contient une Date.
2. **Agrégation :** Commande possède un tableau dynamique de LigneCommande.
3. **Association :** Commande est rattachée à un Client via idClient.
4. **Référence :** LigneCommande référence un Produit du catalogue via refProduit.
5. **Calcul en cascade :** Ligne → Commande → Client → Boutique pour les montants et le chiffre d'affaires.
6. **Cohérence du stock :** toute commande validée décrémente Produit.quantiteStock ; toute annulation la réintègre.

Section 4 - Architecture Modulaire en C

L'architecture modulaire facilite la maintenance et la réutilisabilité du code. Chaque module est compose de deux fichiers :

- Fichier header (.h) : interface publique du module ;
- Fichier source (.c) : implémentation des fonctionnalités.

Structure des modules :

Module	Fichier .h	Fichier .c	Responsabilité
Catalogue produits	produit.h	produit.c	CRUD des produits, gestion du stock et des alertes de réapprovisionnement
Clients	client.h	client.c	CRUD des clients, adresses de livraison, points de fidélité
Commandes	commande.h	commande.c	Création des commandes, gestion des lignes, calcul des montants
Statistiques	statistique.h	statistique.c	Algorithmes de tri, palmarès produits et clients, chiffre d'affaires
Saisie & validation	utils.h	utils.c	Saisie sécurisée, validation des données, affichage formaté
Initialisation	initialisation.h	initialisation.c	Jeu de données de démonstration chargé en mémoire

Rôle des fichiers :

- **Fichiers .h (header) :** déclarations des structures, prototypes de fonctions, constantes (#define), inclusions des bibliothèques nécessaires.
- **types.h :** déclarations des structures partagées (Date, Adresse, Produit, LigneCommande, Commande, Client, Boutique) et des constantes ; inclus par tous les modules pour éviter les inclusions circulaires entre client.h et commande.h.
- **Fichiers .c (source) :** implémentation des fonctions, logique métier, algorithmes de traitement.
- **main.c :** point d'entrée du programme, initialisation de la structure Boutique, boucle principale du menu et libération finale de la mémoire.
- **Makefile :** fortement recommandé pour compiler proprement l'ensemble des modules (gcc -Wall -Wextra).

Section 5 - Approche Progressive

Le développement se fera de manière progressive. Il est indispensable de respecter cet ordre : chaque étape doit être testée avant de passer à la suivante.

1. **Composition :** Client contient une Adresse ; Commande contient une Date.
2. **Agrégation :** Commande possède un tableau dynamique de LigneCommande.
3. **Association :** Commande est rattachée à un Client via idClient.
4. **Référence :** LigneCommande référence un Produit du catalogue via refProduit.
5. **Calcul en cascade :** Ligne → Commande → Client → Boutique pour les montants et le chiffre d'affaires.
6. **Cohérence du stock :** toute commande validée décrémente Produit.quantiteStock ; toute annulation la réintègre.
7. Toute création de commande décrémente le stock ; la suppression d'une ligne d'une commande EN_COURS ou l'annulation réintègre la quantité correspondante.
8. **Suppression logique** : les contrôles d'unicité (référence produit, adresse électronique) portent sur tous les enregistrements, y compris désactivés.
9. **Unicité globale** : les numéros de commande et les identifiants clients sont attribués par les compteurs de Boutique, jamais saisis à la main.
10. **Unicité des lignes** : une commande ne peut pas contenir deux lignes pour la même référence produit ; le contrôle du stock porte sur la quantité cumulée.

Section 6 - Fonctionnalités Attendues

Gestion du catalogue produits

- **Ajouter un produit :** saisie complète avec contrôle d'unicité de la référence.
- **Mettre à jour un produit :** modification du prix, du stock et du seuil d'alerte.
- **Supprimer un produit :** suppression logique, refusée si le produit est utilisé dans une commande non annulée.
- **Afficher la liste :** vue complète du catalogue avec valorisation du stock.
- **Rechercher un produit :** par référence exacte ou par mot-clé de la désignation.

Gestion des clients

- **Ajouter un client :** création avec adresse complète et contrôle d'unicité de l'adresse électronique.
- **Mettre à jour un client :** modification des coordonnées et de l'adresse de livraison.
- **Supprimer un client :** suppression logique, refusée si des commandes non annulées existent.
- **Afficher la liste :** vue d'ensemble avec nombre de commandes et chiffre d'affaires généré.
- **Afficher un client :** fiche détaillée avec l'historique complet de ses commandes.

Gestion des commandes

- **Créer une commande :** sélection du client, saisie de N lignes de produits, contrôle du stock disponible.
- **Ajouter ou supprimer une ligne :** possible uniquement sur une commande au statut EN_COURS.
- **Changer le statut :** progression EN_COURS → EXPEDIEE → LIVREE.
- **Annuler une commande :** passage au statut ANNULEE avec réintégration automatique du stock.
- **Afficher une commande :** détail des lignes, remise appliquée, TVA, frais de port et total TTC.

Affichages et statistiques

- **Chiffre d'affaires :** total HT et TTC de la boutique, et par période (mois, année).
- **Palmarès produits :** top 5 des produits classés par quantité vendue.
- **Palmarès clients :** classement des clients par chiffre d'affaires généré.
- **Alertes de stock :** liste des produits dont le stock est inférieur ou égal au seuil d'alerte.

Section 7 - Spécifications Techniques

IMPORTANT : Pour faciliter les tests et optimiser le temps lors de la soutenance, votre programme DOIT inclure des données pré-initialisées en dur dans votre code (fichier initialisation.c).

Données minimales requises

Votre fichier initialisation.c doit contenir au minimum :

- 8 produits répartis dans au moins 3 catégories, dont 2 en alerte de stock ;
- 5 clients avec adresses complètes et identifiants distincts ;
- 6 commandes couvrant les 4 statuts possibles, dont au moins deux commandes de 3 lignes ou plus et au moins une dépassant le palier de 600 € HT.

Exemple d'initialisation (extrait) :

```c
/* initialisation.c - jeu de données de demonstration */
#include <stdlib.h>
#include <string.h>
#include "initialisation.h"

void initialiserDonneesTest(Boutique *b) {
    /* 1. Catalogue : allocation dynamique de 10 emplacements */
    b->capaciteProduits = 10;
    b->produits = (Produit *)malloc(b->capaciteProduits * sizeof(Produit));
    if (b->produits == NULL) { /* ... gestion d'erreur ... */ }
    b->nbProduits = 0;
    Produit modele[] = {
        {"PRD-0001", "Clavier mecanique",  "Informatique",  79.90,  42, 10, 1},
        {"PRD-0002", "Souris sans fil",    "Informatique",  24.50,   8, 15, 1},
        {"PRD-0003", "Ecran 27 pouces",    "Informatique", 199.00,   6,  5, 1},
        {"PRD-0004", "Casque audio",       "Audio",         89.90,  25, 10, 1},
        {"PRD-0005", "Enceinte Bluetooth", "Audio",         59.90,  12,  8, 1},
        {"PRD-0006", "Cable HDMI 2 m",     "Accessoires",    9.90, 120, 30, 1},
        {"PRD-0007", "Hub USB-C 7 ports",  "Accessoires",   49.90,   4, 10, 1},
        {"PRD-0008", "Disque SSD 1 To",    "Stockage",     109.00,  18,  6, 1}
    };
    for (int i = 0; i < 8; i++)
        b->produits[b->nbProduits++] = modele[i];

    /* 2. Clients : allocation dynamique de 8 emplacements */
    b->capaciteClients = 8;
    b->clients = (Client *)malloc(b->capaciteClients * sizeof(Client));
    if (b->clients == NULL) { /* ... gestion d'erreur ... */ }
    b->nbClients = 0;
    /* ... 5 clients, chacun avec commandes = NULL et capaciteCommandes = 0 ... */

    /* 3. Commande n01 : 3 lignes allouees dynamiquement */
    Commande c1;
    c1.numero       = 1001;
    c1.idClient     = 1;
    c1.dateCommande = (Date){12, 3, 2026};
    strcpy(c1.statut, "LIVREE");
    c1.capaciteLignes = 3;
    c1.lignes = (LigneCommande *)malloc(c1.capaciteLignes * sizeof(LigneCommande));
    c1.nbLignes = 0;
    /* ... ajout des lignes, puis calculCommande(&c1) et rattachement au client ... */
}
/* Liberation complete : une seule fuite memoire est sanctionnee */
void detruireBoutique(Boutique *b) {
    for (int i = 0; i < b->nbClients; i++) {
        for (int j = 0; j < b->clients[i].nbCommandes; j++)
            free(b->clients[i].commandes[j].lignes);
        free(b->clients[i].commandes);
    }
    free(b->clients);
    free(b->produits);
}
```

Avantages de cette approche

Gain de temps considérable lors de la soutenance.
Tests rapides de toutes les fonctionnalités.
Démonstration fluide, sans saisies manuelles.
Données cohérentes et réalistes pour l'évaluation.
Focus sur la présentation plutôt que sur la saisie.

Recommandations

- Créez des fichiers séparés initialisation.h et initialisation.c.
- Commentez clairement vos données de test.
- Assurez-vous de la cohérence des données : références produits valides, identifiants clients existants, stock suffisant.
- Prévoyez également la possibilité d'ajouter manuellement des données pendant la démonstration.
- Testez votre initialisation avant la soutenance.

Contraintes techniques

1. **IMPORTANT : travail en mémoire uniquement (aucune persistance : les données ne sont pas rechargées au démarrage).**
2. **Utilisation exclusive du langage C**
3. **Compilation sans erreur ni avertissement**
4. **Allocation dynamique obligatoire pour toutes les collections : produits, clients, commandes et lignes de commande.**
5. **Libération complète de la mémoire : aucune fuite tolérée (valgrind --leak-check=full).**
6. **Contrôle systématique du retour de malloc et realloc (valeur NULL).**
7. **Validation des saisies utilisateur et gestion explicite des erreurs.**
8. **Code commenté, bien structuré, respect des conventions de nommage.**
9. **IMPORTANT : travail en mémoire uniquement (aucune sauvegarde fichier).**

Section 8 - Exercice de Créativité

Pour enrichir votre projet, vous devez implémenter deux fonctionnalités supplémentaires qui vous semblent pertinentes. Quelques pistes possibles : recherche multi-critères dans le catalogue, export des statistiques au format CSV, gestion des retours et des avoirs, application d'un code promotionnel, statistiques par catégorie, historique des opérations ou mode « démonstration automatique ».

Section 9 - Modalités d'Évaluation

Fonctionnalités	Qualité du code
Toutes les fonctions demandées sont opérationnelles.	Structure, lisibilité, commentaires, respect des conventions.
Architecture	Présentation
Organisation modulaire, séparation des responsabilités, gestion de la mémoire.	Clarté de l'exposé, maîtrise du projet, réponses aux questions.

La gestion de la mémoire (allocations, libérations, absence de fuite) est évaluée au titre des critères « Qualité du code » et « Architecture ».

Section 10 - Conseils et Bonnes Pratiques

Développement

- Commencer par les structures de données avant d'écrire les fonctions.
- Écrire les fonctions d'allocation et de libération dès le début du projet.
- Tester chaque fonction individuellement avant de l'intégrer.
- Utiliser des noms de variables et de fonctions explicites.
- Commenter le code de manière claire et concise.
- Gérer les cas d'erreur, les saisies incorrectes et les retours NULL.

Préparation du projet

- Préparer des données de démonstration réalistes et variées.
- Maîtriser parfaitement toutes les fonctionnalités de votre application.
- Être capable d'expliquer vos choix techniques et algorithmiques.
- Anticiper les questions du jury sur l'architecture et sur l'allocation dynamique.

Présentation

- Utiliser les données pré-initialisées en dur pour ne pas perdre de temps lors de la soutenance.
- Expliquer la logique de votre architecture modulaire.
- Présenter les défis techniques rencontrés et les solutions retenues.
- Montrer la possibilité d'ajouter de nouvelles données en plus des données initialisées.

Section 11 - Dépôt du travail

Premier dépôt obligatoire : à remettre sur Moodle à la fin de votre avant-dernière séance (semaine du 26 octobre 2026). Déposez votre travail réalisé à cette date sous forme d'archive (dossier zippé contenant l'ensemble des fichiers de votre projet) dans l'espace de dépôt correspondant à votre groupe. Le nom de l'archive doit correspondre aux noms des deux personnes formant le binôme, par exemple : DupontMartin_LefebvreClair.zip.Un seul dépôt par binôme. Format impose : Nom1Prenom1_Nom2Prenom2.zip (nom de famille puis prénom, sans espace ni accent).

. Un seul dépôt par binôme.

Deuxième dépôt obligatoire : à remettre sur Moodle au plus tard la veille de votre dernière séance (prevue durant la semaine du 2 Novembre) de XTI101 avant 23h59, sous forme d'archive (dossier zippé contenant l'ensemble des fichiers de votre projet). Utilisez l'espace de dépôt correspondant à votre groupe. Le nom de l'archive doit respecter le même format que le premier dépôt : DupontMartin_LefebvreClair.zip. Un seul dépôt par binôme.